

ABC PHARMACY - SALESFORCE ENTRY ASSESSMENT

ABC Pharmacy on Salesforce

Solution documentation — data model, Apex components, and the reasoning behind them

VERSION	1.0
DATE	14 September 2026
AUTHOR	Youssef Maged — prepared for Cloudastick Systems
STATUS	Issued

Contents

1. 1. Overview and scope

2. 2. Architecture

3. 3. Data model

3.1 Entity relationships

3.2 Objects

3.3 Custom fields

3.4 Validation rules

3.4a Declarative automation

3.5 Design decisions and assumptions

4. 4. Apex components

4.1 Classes

4.2 Test classes and helpers

4.3 Triggers, pages and other components

5. 5. The REST API

6. 6. The warehouse console

6.1 The icon rule, and why precedence had to be decided

6.2 Implementation notes

7. 7. Logging and error handling

8. 8. Archival

8.1 What it does

8.2 The order of operations inside execute()

8.3 Why the writer is injected

8.4 Reading the archive

9. 9. Security model

10. 10. Salesforce features used, and why

11. 11. Deployment and operations runbook

11.1 Deploy into a fresh org

11.2 Seed and demonstrate

11.3 Verified in this org

12. 12. Testing

13. 13. Reviewer access

14. 14. Known limitations and future work

1. Overview and scope

Pharmacy **ABC** needs a Salesforce environment to manage customers, its product catalogue and orders. It runs one warehouse with one operator, and it has a mobile app built in Ionic that must be able to browse the catalogue and build a shopping cart.

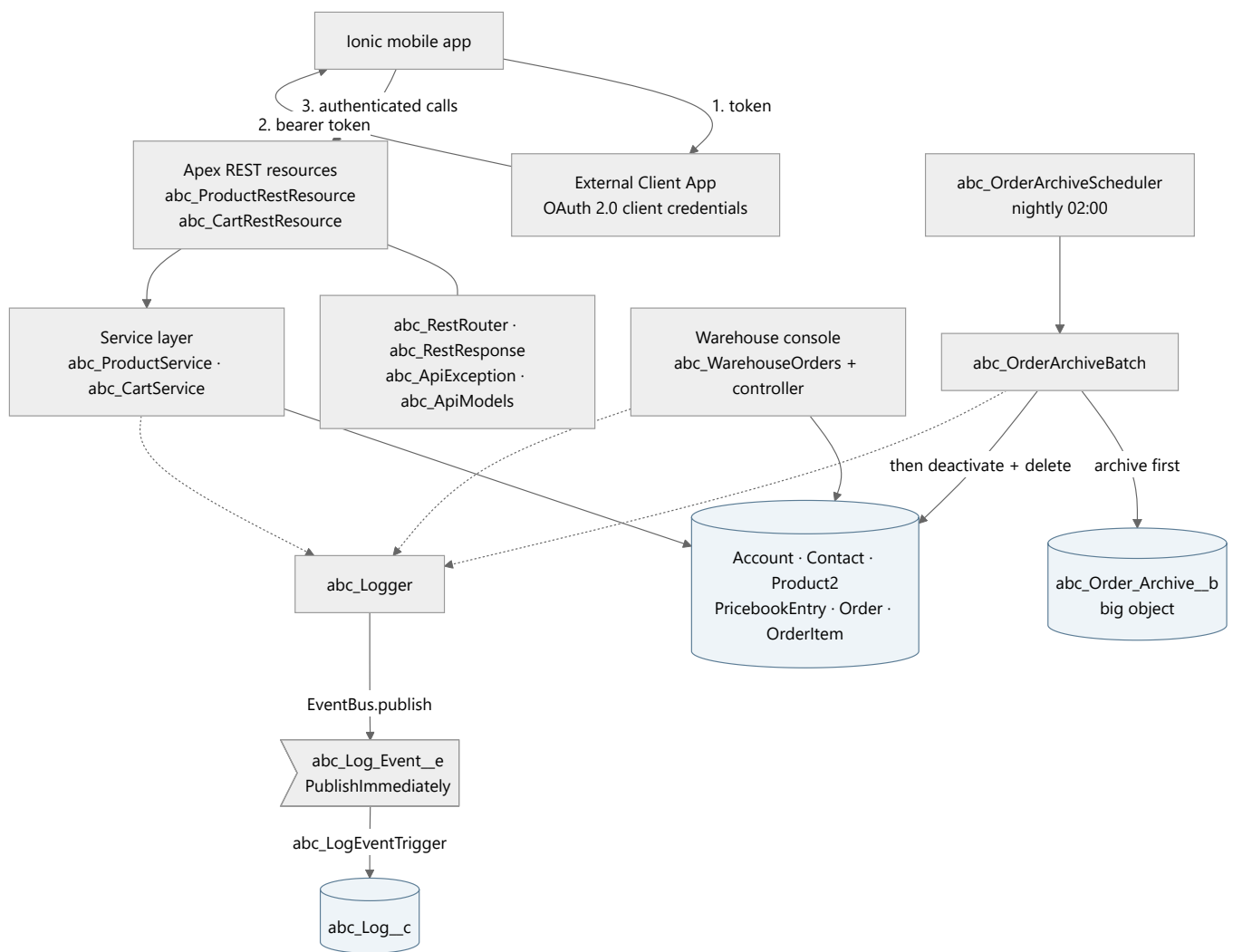
This solution delivers four things:

#	Deliverable	Where it lives
1	A data model and a Lightning app for customers, products and orders	<code>abc_Pharmacy_Management</code>
2	An Apex REST API for the Ionic app, with error handling and logging	<code>/services/apexrest/abc/v1/*</code>
3	A Visualforce console for the warehouse operator	<code>abc_WarehouseOrders</code>
4	A Batch Apex job archiving year-old delivered orders into a Big Object	<code>abc_OrderArchiveBatch</code> → <code>abc_Order_Archive__b</code>

Every API name created for this project is prefixed `abc_`, as the assessment requires. The only exceptions are the standard objects being extended (`Order` , `Product2`) and the standard `OrderStatus` value set, none of which can be renamed.

Out of scope. No payment capture, no delivery routing, no customer-facing community, and no inventory receipting — stock is decremented and corrected, not purchased. Section 12 lists what a production build would add.

2. Architecture



Three ideas hold the design together.

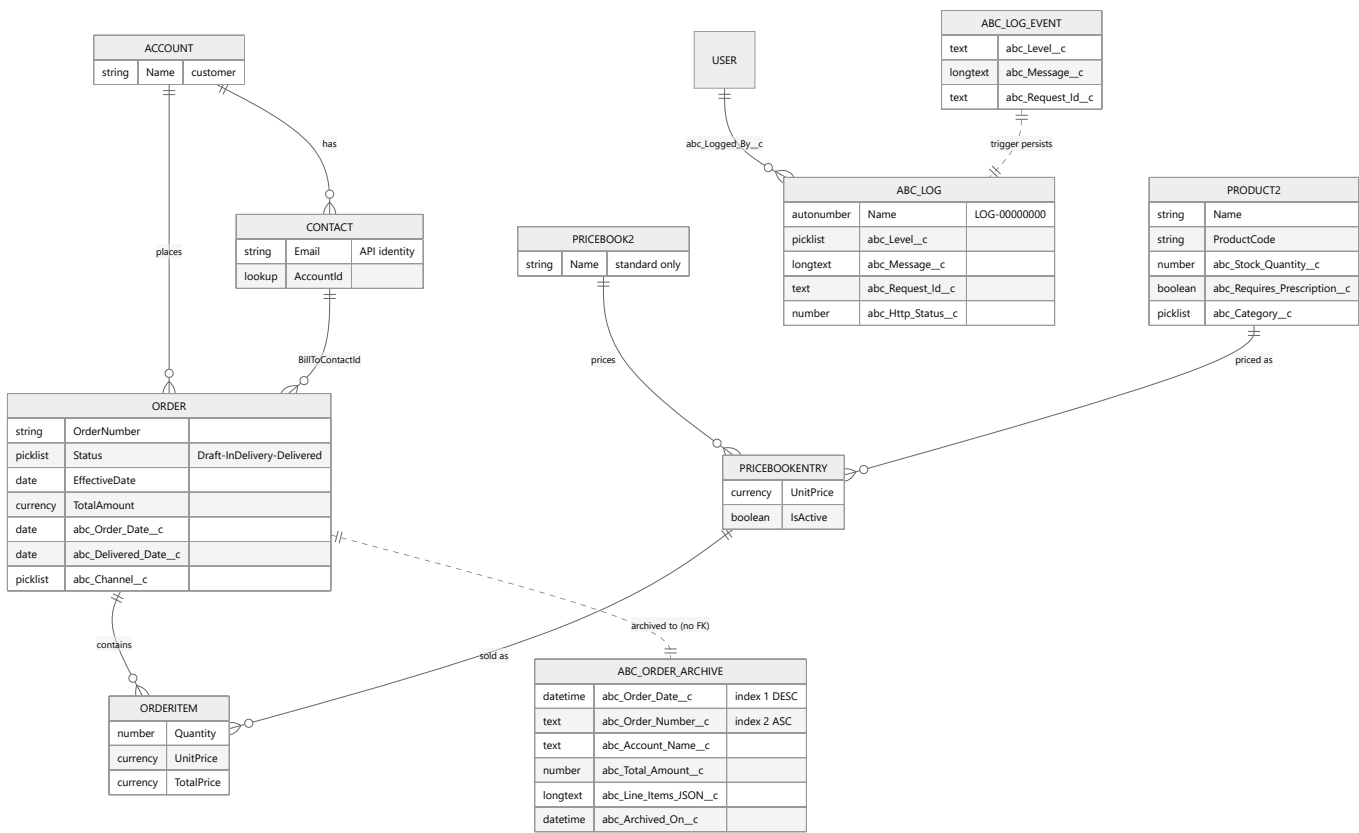
Standard objects first. Salesforce already models customers, catalogues and orders. Re-implementing them as custom objects would throw away the order lifecycle, the price book, `TotalAmount` roll-ups, and every report and page layout that comes with them. The solution extends the standard objects with `abc_`-prefixed fields instead.

A service layer between REST and the database. The two `@RestResource` classes do nothing but read the request, call a service and write the envelope. All rules live in `abc_ProductService` and `abc_CartService`, which can be tested without constructing a `RestContext` and could be reused by a Lightning component or a Flow without touching HTTP.

Logging that survives a rollback. Business code never inserts a log record. It buffers a platform event and publishes it once, and a subscriber turns that into `abc_Log__c`. The reason is in section 7.

3. Data model

3.1 Entity relationships



3.2 Objects

Object	Type	Purpose	Key relationships
<code>Account</code>	Standard	The customer. One per person or household.	Parent of Contact and Order
<code>Contact</code>	Standard	The person. <code>Email</code> is the identity the Ionic app matches on.	Child of Account; <code>Order.BillToContactId</code>
<code>Product2</code>	Standard	The catalogue item.	Parent of PricebookEntry
<code>Pricebook2</code>	Standard	The standard price book only — one price per product.	Parent of PricebookEntry
<code>PricebookEntry</code>	Standard	Ties a product to a price.	Product2 × Pricebook2; referenced by OrderItem
<code>Order</code>	Standard	Both the shopping cart (Draft) and the placed order.	Account, Contact, Pricebook2; parent of OrderItem
<code>OrderItem</code>	Standard	One product line on an order.	Child of Order; references PricebookEntry
<code>abc_Order_Archive__b</code>	Big object	Cold storage for delivered orders older than a year.	None — big objects support no relationships
<code>abc_Log_Event__e</code>	Platform event	Transport for a log entry.	Consumed by <code>abc_LogEventTrigger</code>
<code>abc_Log__c</code>	Custom object	The persisted application log.	Lookup to User

3.3 Custom fields

Product2

Field	Type	Purpose
<code>abc_Stock_Quantity__c</code>	Number(18,0), default 0	Units in the warehouse. Returned by the API so the app can hide or flag what it cannot sell.
<code>abc_Requires_Prescription__c</code>	Checkbox	Lets the app prompt for a prescription upload.
<code>abc_Category__c</code>	Picklist (restricted): Medicine, Supplement, Personal Care, Medical Device	Merchandising grouping and the <code>category</code> API filter.

Order

Field	Type	Purpose
<code>abc_Order_Date__c</code>	Date, default <code>TODAY()</code>	The date the customer placed the order. This is the age the archival batch measures.
<code>abc_Delivered_Date__c</code>	Date	When the order reached the customer.
<code>abc_Channel__c</code>	Picklist (restricted): Manual, Ionic App	Separates app traffic from counter sales. Orders created through the REST API are stamped <code>Ionic App</code> .

`Order.Status` gains two values beyond the platform's Draft and Activated: **In delivery** and **Delivered**. Both are mapped to the **Activated** status code — see section 3.5.

`abc_Order_Archive__b` (big object). Index: `abc_Order_Date__c` DESC, then `abc_Order_Number__c` ASC.

Field	Type	Notes
<code>abc_Order_Date__c</code>	DateTime, required	Index field 1. DateTime because big objects have no Date type.
<code>abc_Order_Number__c</code>	Text(30), required	Index field 2.
<code>abc_Order_Id__c</code>	Text(18)	The id the order held before deletion.
<code>abc_Account_Id__c</code> / <code>abc_Account_Name__c</code>	Text(18) / Text(255)	Customer, denormalised so the archive reads on its own.
<code>abc_Status__c</code> / <code>abc_Channel__c</code>	Text(40)	Snapshot at archival time.
<code>abc_Effective_Date__c</code> / <code>abc_Delivered_Date__c</code>	DateTime	
<code>abc_Total_Amount__c</code>	Number(16,2)	Number, not Currency — big objects have no Currency type.
<code>abc_Line_Items_JSON__c</code>	LongTextArea(131072)	The order's lines as JSON.
<code>abc_Archived_On__c</code>	DateTime	When the batch wrote the row.

`abc_Log__c` — name is an auto-number `LOG-{{00000000}}` ; fields `abc_Level__c` (DEBUG/INFO/WARN/ERROR), `abc_Source__c` (REST/Batch/Trigger/Visualforce/Other), `abc_Message__c` , `abc_Stack_Trace__c` , `abc_Class__c` , `abc_Method__c` , `abc_Record_Id__c` , `abc_Request_Id__c` , `abc_Quiddity__c` , `abc_Http_Status__c` , `abc_Timestamp__c` , `abc_Logged_By__c` (lookup to User). `abc_Log_Event__e` carries the same fields, with the user as text (`abc_Logged_By_Id__c`) because platform events cannot hold lookups.

3.4 Validation rules

Object	Rule	Why
Product2	<code>abc_Stock_Quantity_Not_Negative</code>	A warehouse cannot hold negative units. Guards manual edits and the trigger-driven decrement.
Order	<code>abc_Order_Date_Not_In_Future</code>	An order cannot be placed in the future; keeps the one-year archival window honest.
Order	<code>abc_Effective_Date_Not_Before_Order_Date</code>	An order cannot take effect before it was placed. Both dates appear side by side on the warehouse console, so an inverted pair would be visibly wrong.

3.4a Declarative automation

One record-triggered flow, `abc_Order_Set_Delivered_Date` : a **before-save** flow on Order that stamps `abc_Delivered_Date__c` with today when an order first reaches Delivered without one. Before-save is the important part — the field is set on the record already being written, so there is no second DML and no recursion to guard against.

3.5 Design decisions and assumptions

A cart is a Draft Order, not a separate object. Salesforce already gives an order a draft state in which lines can be added and removed freely. Introducing an `abc_Cart__c` object would mean copying data at checkout, and a copy can fail halfway. Here, checkout is a status change on the record that already exists: nothing moves, and there is no window in which a cart and its order can disagree. It also means the warehouse sees a live cart in the same list as everything else.

Account + Contact rather than Person Accounts. Person Accounts are not enabled in a Developer Edition org by default and cannot be turned off once on. A retail pharmacy customer maps cleanly to an Account with one Contact, and the Ionic app identifies people by email, which lives on Contact.

One price book. ABC has a single retail price per product. Using only the standard price book removes an entire dimension of configuration, and `Order.Pricebook2Id` still has to be set for line items to be addable at all.

`abc_Order_Date__c` is distinct from the standard `EffectiveDate`. The assessment asks the warehouse console to show the *effective date*, and asks the batch to age orders by *Order Date*. Those are different business facts — when the order takes commercial effect versus when the customer placed it — so they are different fields. Conflating them would make the archival rule depend on a date the warehouse can edit.

Custom statuses are mapped to the Activated status code. Salesforce groups every order status under one of two codes, Draft or Activated. "In delivery" and "Delivered" are both mapped to Activated, which is what makes the platform freeze the line items once fulfilment has begun — a picker cannot quietly add a product to an order that is already on a van. The cost is that an activated order cannot be deleted until its status is set back to a Draft-coded value, which is exactly what the archival batch does before deleting.

The archive is one big-object row per order, with line items as JSON. Big objects support neither master-detail nor lookup relationships, so a parent/child archive is not available. Two rows for one order could also be written half-successfully. Keeping the lines inside the parent row makes an archived order atomic: one `insertImmediate` either archives the whole order or none of it.

The archive index is `[Order Date DESC, Order Number ASC]`. Big object queries must filter the index fields in index order, so the first field decides how the archive can be browsed — by period, which is how anyone looks for an old order. Order number makes the pair unique, which means re-running the batch over an already-archived order overwrites the same row instead of duplicating it. **A big object index cannot be changed after deployment**, so this was settled before the first deploy.

Field-level security is delivered as permission sets, never profile edits. Fields deployed through the Metadata API arrive with no FLS for anybody, including a System Administrator. Granting it through three named permission sets keeps the solution installable into an org whose profiles are already managed by someone else.

4. Apex components

4.1 Classes

Class	Type	Responsibility	Tested by
<code>abc_Logger</code>	Logging	Buffers <code>abc_Log_Event__e</code> entries and publishes them in one flush. Captures request id, quiddity, user and HTTP status; truncates rather than losing data; never throws.	<code>abc_LoggerTest</code>

Class	Type	Responsibility	Tested by
<code>abc_LogEventTriggerHandler</code>	Logging	Turns delivered events into <code>abc_Log__c</code> rows. Partial-success insert, and an unusable user id is dropped rather than failing the batch.	<code>abc_LoggerTest</code>
<code>abc_ApiException</code>	REST	The one exception the API throws on purpose. Owns the error-code → HTTP-status contract.	<code>abc_RestResponseTest</code>
<code>abc_RestResponse</code>	REST	The single writer of the response envelope. Uses <code>JSONGenerator</code> so <code>data: null</code> and <code>errors: []</code> are always present.	<code>abc_RestResponseTest</code>
<code>abc_ApiModels</code>	REST	The wire contract: request classes for strict parsing, DTOs for responses. No SObject ever reaches the client.	(exercised throughout)
<code>abc_RestRouter</code>	REST	Normalises <code>requestURI</code> — apexrest prefix, trailing slashes, query strings — and reads typed query parameters.	<code>abc_RestRouterTest</code>
<code>abc_ProductService</code>	Service	The catalogue: search, category, stock and paging, as a bind-variable-only dynamic query.	<code>abc_ProductServiceTest</code>
<code>abc_CartService</code>	Service	Cart lifecycle: find-or-create customer, create, read, add item with quantity merge, checkout.	<code>abc_CartServiceTest</code>
<code>abc_ProductRestResource</code>	REST	<code>GET /abc/v1/products</code> .	<code>abc_ProductRestResourceTest</code>
<code>abc_CartRestResource</code>	REST	The four cart routes, dispatched from one <code>@HttpPost</code> and one <code>@HttpGet</code> .	<code>abc_CartRestResourceTest</code>
<code>abc_WarehouseOrdersController</code>	Visualforce	Reads every order with its line items and resolves each row's icon.	<code>abc_WarehouseOrdersControllerTest</code>
<code>abc_IOrderArchiver</code>	Batch	The seam between the batch and the big object. Exists so the batch is testable — see section 8.3.	— (interface)
<code>abc_BigObjectOrderArchiver</code>	Batch	The production writer: <code>Database.insertImmediate</code> .	<code>abc_BigObjectOrderArchiverTest</code>
<code>abc_OrderArchiveBatch</code>	Batch	Selects, maps, archives, deactivates, deletes, and reports. <code>Database.Stateful</code> .	<code>abc_OrderArchiveBatchTest</code>
<code>abc_OrderArchiveScheduler</code>	Batch	<code>Schedulable</code> wrapper for the nightly run.	<code>abc_OrderArchiveSchedulerTest</code>

4.2 Test classes and helpers

Class	Purpose
<code>abc_TestDataFactory</code>	Shared fixtures. Encapsulates the awkward parts of the order model in one place: insert as Draft with a price book, attach items, then move status; and <code>Test.setCreateDate</code> for backdating.
<code>abc_FakeOrderArchiver</code>	Test double for <code>abc_IOrderArchiver</code> . Records rows, can fail selected orders, throw, or return a mismatched result count.
<code>abc_LoggerTest</code> · <code>abc_RestResponseTest</code> · <code>abc_RestRouterTest</code> · <code>abc_ProductServiceTest</code> · <code>abc_CartServiceTest</code> · <code>abc_ProductRestResourceTest</code> · <code>abc_CartRestResourceTest</code> · <code>abc_WarehouseOrdersControllerTest</code> · <code>abc_OrderArchiveBatchTest</code> · <code>abc_BigObjectOrderArchiverTest</code> · <code>abc_OrderArchiveSchedulerTest</code>	See section 8.

4.3 Triggers, pages and other components

Component	Type	Notes
<code>abc_LogEventTrigger</code>	Trigger on <code>abc_Log_Event__e</code> (after insert)	One line; all logic in the handler so it is directly unit testable.
<code>abc_WarehouseOrders</code>	Visualforce page	SLDS, <code>apex:repeat</code> , ~25 lines of plain JavaScript for row expansion.
<code>abc_Warehouse_Orders</code>	Custom tab	Surfaces the page in the Lightning app.
<code>abc_Log__c</code>	Custom tab	With list views <code>abc Recent Errors</code> and <code>abc All Logs</code> .
<code>abc_Pharmacy_Management</code>	Lightning app	Home, Accounts, Contacts, Products, Orders, Warehouse Orders, abc Logs.
<code>abc_Order_Set_Delivered_Date</code>	Record-triggered Flow	Before-save on Order: when an order first reaches Delivered with no delivered date, stamps today.
<code>abc_Pharmacy_Admin</code> · <code>abc_Warehouse_User</code> · <code>abc_API_Integration</code>	Permission sets	Section 9.

5. The REST API

Full reference, with request and response examples, is in `API_DOCUMENTATION.md`. In summary:

Method and path	Purpose	Success
GET /abc/v1/products	Catalogue, with search, category, includeOutOfStock, limit, offset	200
POST /abc/v1/carts	Create a cart; find or create the customer by email	201
GET /abc/v1/carts/{cartId}	Read a cart	200
POST /abc/v1/carts/{cartId}/items	Add a product, merging quantity if it is already in the cart	200
POST /abc/v1/carts/{cartId}/checkout	Place the order	200

Every response, success or failure, uses one envelope:

```
{ "success": true, "data": { }, "errors": [], "requestId": "4Zx..." }
```

The client parses one shape and never branches on status code to find the payload. `requestId` is the platform request id that `abc_Logger` stamps on every log entry raised in the same transaction, so a request id quoted in a support ticket resolves directly to the rows that explain it.

Three implementation choices are worth calling out.

Strict JSON parsing. Bodies are read with `JSON.deserializeStrict` against explicit request classes. A misspelled attribute — `quantityy` instead of `quantity` — is rejected as `MALFORMED_JSON` rather than being silently ignored and then failing later as "quantity is required", which is far harder for a mobile developer to diagnose.

Routing on the path. Apex REST allows one `@HttpPost` and one `@HttpGet` per class, so `abc_CartRestResource` dispatches its four routes on the part of the URI after the base path. `abc_RestRouter` absorbs the fact that `requestURI` arrives with or without the `/services/apexrest` prefix, with or without a trailing slash, and sometimes with a query string still attached. An unrecognised path returns 404 `ROUTE_NOT_FOUND` listing the valid routes, rather than falling through to something surprising.

Queries are bound, never concatenated. `abc_ProductService` builds its SOQL dynamically to apply optional filters, but every caller value goes in through `Database.queryWithBinds`. A test asserts that a search term containing quote characters returns no rows instead of altering the query.

6. The warehouse console

`abc_WarehouseOrders` lists every order — number, status, effective date, customer, total — with an icon before the order number and a row that expands to show the products to pick, with quantity and unit price.

6.1 The icon rule, and why precedence had to be decided

The assessment specifies four icons:

Condition	Icon
Order created today	🕒 hourglass with flowing sand
Order created before today	🕒 hourglass done
Status "In delivery"	🚚 delivery truck
Status "Delivered"	✅ white heavy check mark

These rules overlap: an order that is Delivered was also created on some day, so a status rule and a date rule both match the same record. The requirement does not say which wins, so the decision is made once, in

`abc_WarehouseOrdersController`, and **stated in a legend on the page itself**:

Delivered → ✅; else In delivery → 🚚; else created today → 🕒; else 🕒.

Status beats date because of what the operator uses the list for. Fulfilment state is the actionable signal — is this mine to pick, is it on the road, is it finished — while the age of the order is context. An order delivered this morning is finished business, and showing it as "created today" would hide that. The rule is total and mutually exclusive, so every row gets exactly one icon, and the status is also spelled out in its own column so the icon is never the only carrier of meaning.

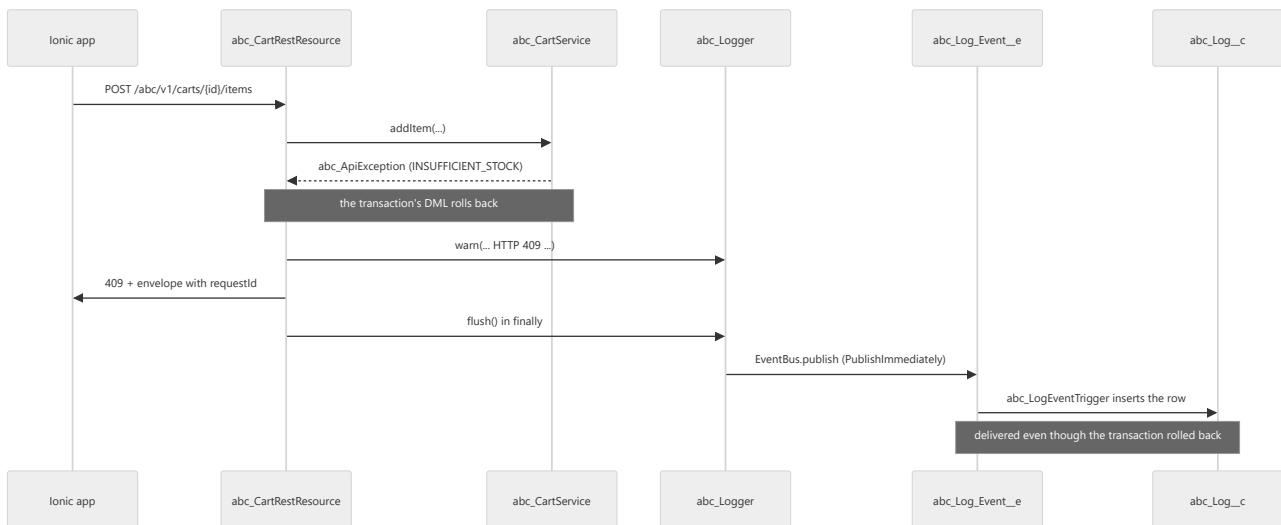
6.2 Implementation notes

The emoji are emitted as **HTML numeric character entities** (`⏳`, `⌛`, `🚚`, `✅`) rather than literal characters. Visualforce markup, the Metadata API and the file encodings in between all handle ASCII entities predictably, whereas a literal astral-plane character such as the truck can survive or not survive a round trip depending on which tool touched the file last.

Row expansion is one delegated click listener in plain JavaScript — no library, no inline `onclick` — so nothing trips the org's content security policy. The toggle maintains `aria-expanded` and `aria-controls`.

`apex:repeat` renders at most 1000 items, so the query is capped at 1000 and ordered newest first. When the cap is reached the page says so rather than silently truncating the warehouse's work list.

7. Logging and error handling



Business code never inserts `abc_Log__c`. It calls `abc_Logger.debug/info/warn/error(...)`, which buffers an `abc_Log_Event__e`, and the handler calls `abc_Logger.flush()` once in a `finally` block.

Why the indirection. The transaction you most need a log for is the one that failed and rolled back — and an ordinary `insert` would roll back with it, destroying the evidence. `abc_Log_Event__e` is declared `PublishImmediately`, which means the platform delivers it regardless of what the surrounding transaction goes on to do.

Other properties worth noting:

- **Correlation.** Every entry carries `Request.getCurrent().getRequestId()`, and the API returns the same value in every response envelope.
- **Levels and source.** Client mistakes (4xx) are logged as WARN, server faults (5xx) as ERROR, so `abc Recent Errors` is not drowned in user typos.
- **Nothing leaks.** A 500 tells the caller only that the request could not be completed, and to quote the requestId. The exception type, message and stack trace stay in `abc_Log__c`.
- **The logger never throws.** A failure to log must not become the error the caller sees; publish failures fall back to the debug log.
- **Bounded.** The buffer caps at 250 entries per transaction, so a runaway loop cannot turn one failed request into a platform-event limit breach.

8. Archival

8.1 What it does

`abc_OrderArchiveBatch` selects every Order whose `Status` is **Delivered** and whose `abc_Order_Date__c` is **more than one year old**, writes one `abc_Order_Archive__b` row per order with its line items serialised to JSON, and then removes the order from Salesforce.

`abc_OrderArchiveScheduler` runs it nightly at 02:00 under the job name `abc_Order_Archive_Nightly` — outside the warehouse's working day and away from the app's traffic, because the job deletes records.

8.2 The order of operations inside `execute()`

The three steps are in a specific and non-obvious order:

1. **Archive first.** `Database.insertImmediate` is treated by the platform as a callout. If any sObject DML — or an `EventBus.publish` — has already happened in the transaction, it fails with "uncommitted work pending".
2. **Then set the status back to Draft and delete.** An Activated-coded order cannot be deleted, and Delivered is Activated-coded.
3. **Flush the log last,** because publishing events is itself DML.

Failures are isolated per order: a row whose archive write fails is left in place and logged, and the others still go. Because the archive is keyed on `(order date, order number)`, a re-run overwrites the same row rather than duplicating it, so retrying is safe.

8.3 Why the writer is injected

Salesforce does not allow big object DML inside an Apex test. `Database.insertImmediate` from a test context raises `System.UnexpectedException: Internal Salesforce.com Query Error` — verified against this org, both inside and outside a `Test.startTest()` boundary — and that exception **cannot be caught**, so the call cannot even be wrapped and tolerated.

The batch has to do both a big object write and ordinary sObject DML in one transaction, so without a seam none of its behaviour could be tested at all. `abc_IOrderArchiver` is that seam: production runs `abc_BigObjectOrderArchiver`, tests run `abc_FakeOrderArchiver`, and the real write is proved against the org by `scripts/apex/abc_run_archive_now.apex` followed by `scripts/apex/abc_query_archive.apex` — a documented step in the runbook, and one that has been run (section 11).

8.4 Reading the archive

Big object SOQL is not ordinary SOQL, and `abc_query_archive.apex` shows the three rules it has to obey: filter the index fields **in index order**, no aggregate functions (`SELECT COUNT()` fails outright), and no `!=` or `LIKE`. The idiom for "everything" is a wide range filter on the first index field.

9. Security model

Access is delivered entirely through permission sets. No profile is modified.

Permission set	For	Objects	Notable permissions
<code>abc_Pharmacy_Admin</code>	Business administrators	Full CRUD on Account, Contact, Product2, Order, OrderItem, <code>abc_Log__c</code> ; read/create/delete on the archive	Activate Orders, Edit Activated Orders, API Enabled; all <code>abc_</code> classes; both tabs; the app
<code>abc_Warehouse_User</code>	The warehouse operator	Read + Edit on Order and OrderItem; Edit on <code>Product2.abc_Stock_Quantity__c</code> ; read-only on customers and the rest of the catalogue; no delete anywhere; no access to the log	Edit Activated Orders (+ Activate Orders, which Salesforce requires as its prerequisite); the Visualforce page and its controller only
<code>abc_API_Integration</code>	The Ionic app's run-as identity	Create/edit Account, Contact, Order, OrderItem; read Product2 and pricing; edit stock	API Enabled, Activate Orders; the REST classes and what they call — no Visualforce, no log, no delete except order lines

The warehouse set is the interesting one: it grants exactly what the assessment describes — "view the orders to commence the outbound deliveries and quantity change updates" — and nothing more. It cannot create an order, cannot delete anything, and cannot see the application log. `ActivateOrder` appears only because Salesforce refuses `EditActivatedOrders` without it; since the operator cannot create orders, activation in practice only ever means moving an order the pharmacy already took into "In delivery" and then "Delivered". That constraint is recorded inline in the permission set file.

Apex enforces the same boundary rather than relying on the UI: services are `with sharing`, SOQL runs `WITH USER_MODE`, and DML runs at `AccessLevel.USER_MODE`. `abc_WarehouseOrdersControllerTest` proves it, by running the controller inside `System.runAs` for a Standard User holding nothing but `abc_Warehouse_User`.

The one deliberate exception is `abc_LogEventTriggerHandler`, which is `without sharing`: the platform-event subscriber runs as the Automated Process user, and a log must be written regardless of who raised it. It copies a payload and reads no business data.

10. Salesforce features used, and why

Feature	Where	Why this rather than something else
Standard Order model (Order, OrderItem, Pricebook)	The core	Brings the draft/activated lifecycle, <code>TotalAmount</code> roll-ups and line-item locking for free. Custom objects would mean rebuilding all of it.
Lightning App	<code>abc_Pharmacy_Management</code>	Puts the business user's six screens behind one navigation bar instead of leaving them to hunt through the app launcher.
Permission Sets	Three	Additive and removable. A profile edit is a change to the org that is hard to reverse and hard to review.
Validation Rules	Three	Data integrity enforced by the platform, so it applies to the UI, the API and the data loader alike — nobody can route around it.
Apex REST	Two resources	The Ionic app needs endpoints shaped around a cart, not around SOBJects. Composite/standard REST would have made the app orchestrate order → price book → line items itself.

Feature	Where	Why this rather than something else
External Client App + OAuth 2.0	<code>abc_Ionic_App</code>	The current way to register an API client; classic Connected Apps are disabled for new orgs. Client credentials keeps the reviewer's setup to one token call.
Platform Events	<code>abc_Log_Event__e</code>	The only mechanism that gets a log out of a transaction that then rolls back.
Custom Object	<code>abc_Log__c</code>	Makes logs reportable, list-viewable and queryable by requestId — a debug log is none of those.
Big Object	<code>abc_Order_Archive__b</code>	Exactly the requirement, and the right tool: unlimited, cheap, indexed storage for records you keep but rarely read.
Batch Apex + Schedulable	<code>abc_OrderArchiveBatch</code>	Archival is unbounded over time; batch chunking is what keeps it inside governor limits as the order history grows.
Record-Triggered Flow	<code>abc_Order_Set_Delivered_Date</code>	A before-save flow is the cheapest thing on the platform that can default one field on the record being saved — no query, no DML, no Apex to maintain. Keeping it declarative also lets the pharmacy change the rule without a deployment. Apex would have been the wrong tool.
Visualforce + SLDS	<code>abc_WarehouseOrders</code>	Requested explicitly. <code>apex:slds</code> keeps it visually native to Lightning without hand-written CSS.
Custom Tabs	Two	Surfaces the page and the log inside the app.

11. Deployment and operations runbook

11.1 Deploy into a fresh org

```
sf org login web --alias abcDev --set-default
sf project deploy start --source-dir force-app
sf org assign permset --name abc_Pharmacy_Admin
sf org assign permset --name abc_Warehouse_User
sf org assign permset --name abc_API_Integration
```

One thing to know before the second deploy: a **scheduled Apex job blocks Apex deployments**. Once `abc_Order_Archive_Nightly` exists, Salesforce refuses to deploy any Apex class while that job is pending — and because a deploy is all or nothing, one scheduled job blocks the whole source directory, not just the batch. `force-app/main/default/settings/Deployment.settings-meta.xml` turns on *Allow deployments of components when corresponding Apex jobs are pending or in progress*, which is the supported remedy; it is deployed as part of the source, so a first deploy into a fresh org sets it before anything is scheduled. The alternative — unschedule before every deploy, reschedule afterwards — is a trap for whoever deploys next.

Two settings must be enabled in Setup by hand — neither is exposed to the Metadata API:

1. **User Interface** → "Set Audit Fields upon Record Creation", so the seed script can backdate `CreatedDate` and the console can show an order created before today. Until it is on, the `CreateAuditFields` permission does not exist in the org at all — which is why `abc_Pharmacy_Admin` grants it and `abc_seed_data.apex` writes the field through `put()` rather than as a field assignment, the static form being a compile error while the setting is off.

2. **Deliverability** → **All email**, so the reviewer's password reset can be sent.

And the API client is registered in **Setup** → **External Client App Manager**: name `abc_Ionic_App`, Local distribution, OAuth enabled, callback `https://localhost/callback`, scope `api`, **Client Credentials Flow** enabled, and under **Policies** a **Run As** user holding `abc_API_Integration`. The consumer key and secret are revealed on the app's Settings tab.

11.2 Seed and demonstrate

```
sf apex run --file scripts/apex/abc_deactivate_sample_products.apex # retire the DE sample catalogue
sf apex run --file scripts/apex/abc_seed_data.apex                  # catalogue, customers, 5 orders
sf apex run --file scripts/apex/abc_schedule_archive.apex          # nightly job at 02:00
sf apex run --file scripts/apex/abc_run_archive_now.apex           # archive now
sf apex run --file scripts/apex/abc_query_archive.apex            # read the archive back
sf apex run --file scripts/apex/abc_seed_archivable_order.apex     # re-arm the demo
```

The seed creates five orders chosen to exercise every behaviour: an open Draft cart, an Activated order, one In delivery, one Delivered recently, and one Delivered over a year ago for the batch to find.

Run `abc_deactivate_sample_products.apex` first, and in every fresh org. `abc_ProductService` defines the catalogue as "active product with an active entry in the standard price book" — the correct definition for a real org, and one that lets a new pharmacy line be added without a code change. A Developer Edition org, however, ships with its own sample catalogue of GenWatt generators, SLAs and Installations, and those records satisfy that definition too. They stay out of the default listing only because their `abc_Stock_Quantity__c` is null and `null > 0` is false in SOQL; call `GET /abc/v1/products?includeOutOfStock=true` and a pharmacy starts advertising diesel generators. Deactivating them fixes the data rather than narrowing the service to a product-code prefix.

11.3 Verified in this org

Check	Result
<code>sf project deploy start --source-dir force-app</code>	Succeeds — 92/92 components, 0 errors, with the nightly job scheduled
<code>sf apex run test --test-level RunLocalTests --code-coverage</code>	133 tests, 100% passing, 91% org-wide coverage
Order status codes	<code>Draft</code> → <code>Draft</code> ; <code>Activated</code> , <code>In delivery</code> , <code>Delivered</code> → <code>Activated</code>
Warehouse console	Renders with legend; all four icons correct against seeded data — ⌚ created today, ⌚ created earlier, 🚚 in delivery, ✅ delivered
Archival batch	<code>AsyncApexJob</code> Completed, 0 errors; order 00000104 archived with all three line items as JSON and deleted from Order; the recent Delivered order untouched
Nightly schedule	<code>abc_Order_Archive_Nightly</code> , <code>0 0 2 * * ?</code> , state WAITING
Delivered-date flow	Moving an order to Delivered with no delivered date stamps today; verified against the org

12. Testing

133 tests, 100% passing, 91% org-wide coverage.

Class	Coverage
<code>abc_ApiException</code> · <code>abc_ApiModels</code> · <code>abc_LogEventTrigger</code> · <code>abc_LogEventTriggerHandler</code> · <code>abc_OrderArchiveScheduler</code> · <code>abc_WarehouseOrdersController</code>	100%
<code>abc_RestRouter</code>	98%
<code>abc_RestResponse</code>	96%
<code>abc_ProductService</code>	95%
<code>abc_Logger</code>	92%
<code>abc_CartService</code>	91%
<code>abc_OrderArchiveBatch</code>	86%
<code>abc_ProductRestResource</code>	82%
<code>abc_CartRestResource</code>	80%
<code>abc_BigObjectOrderArchiver</code>	75% — see below

The tests are written to assert behaviour rather than implementation: error cases assert the specific error **code**, because those codes are a published contract with the Ionic developers and a silent change from 409 to 400 would break them.

A few tests exist specifically to pin platform behaviour the design depends on — that Salesforce refuses to activate an order with no products, that a quote character in a search term cannot alter the SOQL, that a cart which has left Draft is frozen.

`abc_BigObjectOrderArchiver` sits at 75% because its single `Database.insertImmediate` statement is unreachable from a test (section 8.3). That line is proved against the org instead.

13. Reviewer access

A System Administrator user has been created for Cloudastick:

Username	<code>assessments@cloudastick.com.orgfarm-ad027e59fe</code>
Email	<code>assessments@cloudastick.com</code>
Profile	System Administrator
Licence	Salesforce
Permission sets	<code>abc_Pharmacy_Admin</code> , <code>abc_Warehouse_User</code>

The password is supplied in the submission email rather than written here, and deliberately not committed to this repository. It was set with `System.setPassword` , not `System.resetPassword` , so there is no 24-hour link to race and no forced change on first login.

On the first login from a new location Salesforce will email a verification code to

`assessments@cloudastick.com`. That is by design - the assessment asks for a reviewer account that works **without** access to the author's personal verification codes, and this account's codes go to Cloudastick's own mailbox.

Once in: open the **ABC Pharmacy Management** app, then the **Warehouse Orders** tab for the console and **abc Logs** for the application log.

The Postman collection in `postman/` covers every endpoint. Import the collection and the environment, set the consumer key and secret, and run the **Get token** request first — everything else picks the token up automatically.

14. Known limitations and future work

Limitations of this build

- **Big object writes are untestable in Apex.** Covered in section 8.3; the real write is verified by a runbook script instead.
- **Seeding a "created before today" order needs an org setting.** The ⏳ rule keys on `CreatedDate`, which can only be written when *Set Audit Fields upon Record Creation* is enabled — it is, in this org, and the seed data exercises all four icons. In an org where it is off, `abc_seed_data.apex` detects that and carries on without backdating, so every seeded order reads as created today. The rule itself is unit tested independently with `Test.setCreatedDate`.
- **Client-credentials OAuth.** Chosen so a reviewer needs one token call. A production Ionic app should use Authorization Code with PKCE, so that actions are attributed to the real customer rather than to a shared integration user.
- **No stock decrement on checkout.** Stock is validated against the cart but not reserved, so two customers can both be told the last unit is available. A production build would decrement on activation and hold stock for the life of a cart.
- **The warehouse console lists 1000 orders.** Enough for a single-warehouse pharmacy; beyond that it needs paging or a date filter.
- **Developer Edition constraints.** Two Salesforce licences remain free, which is enough for the reviewer and one warehouse user. Salesforce Platform licences cannot access Order or Product2 at all, so a real warehouse-only user would still need a full licence.

What a production build would add

- Authorization Code + PKCE, or a backend-for-frontend that holds the secret.
- Stock reservation, with a cart expiry that releases it.
- Prescription upload and pharmacist approval for `abc_Requires_Prescription__c` products.
- Delivery routing and driver assignment, replacing the single "In delivery" status with real fulfilment tracking.
- A retention policy for `abc_Log__c` — platform-event logging is cheap to write and will grow without one.
- Rate limiting on the API, and idempotency keys on checkout.